



Module Code & Module Title

CC5067NT Smart Data Discovery

60% Individual Coursework

Submission: Final Milestone

Academic Semester: Spring Semester 2026

Credit: 15 credit semester long module

Student Name: Manoj Katuwal

London Met ID: 24045922

College ID: np05cp4a240306

Assignment Due Date: May 10, 2026

Assignment Submission Date: May 11 , 2026.

Submitted To: Nikesh Regmi

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1	Introduction.....	1
1.1	Aims and Objective.....	2
1.2	Tools and Technologies used	3
1.2.1	Programming Language:.....	3
1.2.2	Libraries Used.....	4
2	Data Understanding	9
2.1	Dataset description.....	9
3	Data Preparation.....	11
3.1	Importing Datasets	11
3.2	Displaying the last 50 records.....	11
3.3	Creating a separate DataFrame with selected columns.....	12
3.4	Handling Missing values (NaN)	13
3.5	Checking Duplicate Records.....	14
3.6	Displaying total number of columns and column names.....	15
3.7	Renaming Label name	16
4	Data Analysis	17
4.1	Summary Statistics of selected two numeric values.....	17
4.2	Correlation analysis of numeric values.....	18
5	Data Exploration	20
5.1	Bar char visualization and Balance Analysis of Label.....	20
5.2	Analysis of Average Flow Duration and Packet Length Mean by Label Categories 22	
5.3	Boxplot of Fwd Packet Length Mean by Traffic Type	24
5.4	Hypothesis testing.....	25
6	Conclusion	26
7	References.....	27

Table of figures

Figure 1:Python logo.....	3
Figure 2:pandas logo.....	4
Figure 3:NumPy logo.....	5
Figure 4:matplotlib logo	6
Figure 5:SciPy logo.....	7
Figure 6:Seaborn logo.....	8
Figure 7:Importing libraries.....	11
Figure 8:Code to display 50 records	11
Figure 9: Python code code of separate Dataframe with selected columns.....	12
Figure 10:Output of showing DataFrame with selected columns.....	12
Figure 11:Python code of checking handling value.....	13
Figure 12:Output of handling value.....	13
Figure 13:Python code of removing NaN values.....	14
Figure 14:Output of showing removing Nan values.....	14
Figure 15:Python code of checking duplicate records	14
Figure 16:Output of showing total number of duplicate records.....	15
Figure 17:Python code of displaying total number of columns and column names.....	15
Figure 18:Output of displaying total number of columns and column names.....	15
Figure 19:Python code to rename label name.....	16
Figure 20:Output of renaming label name	16
Figure 21: Python code to display summary Statistics	17
Figure 22:Output of summary statistics	18
Figure 23:Python code to display correlation analysis of numeric values.....	18
Figure 24:Output showing a correlation analysis	19
Figure 25:Python code to visualize bar chart and balance analysis of label.....	20
Figure 26:Output of showing visualize bar chart and balance analysis of label.....	21
Figure 27:Code to display average flow duration and packet length mean	22
Figure 28:Output showing average flow duration and packet length mean	23
Figure 29:Code to display Fwd Packet Length mean by traffic type	24
Figure 30:Code of hypothesis testing	25

Figure 31:Output of hypothesis testing.....	25
---	----

Table of tables

Table 1: Dataset table.....	9
-----------------------------	---

1 Introduction

As network-based systems are rapidly growing, network security has been given due consideration. The ability to identify abnormal or malicious network activities, otherwise referred to as cyber-attacks, is one of the main issues in this field. The analysis of data is a very important step in defining such patterns, as well as enhancing intrusion detection systems.

The project is concerned with analyzing a network traffic dataset which includes different features including flow duration, number of packets, packet length and statistical values as well as a label of whether the traffic is normal or an attack. The primary aim of this paper is to learn and investigate the nature of network traffic and define the patterns that would assist in distinguishing between normal and malicious traffic.

It is analyzed with the help of Python and its data analysis packages, such as pandas, numpy, matplotlib, and seaborn. Different processes like data discovery, data preparation, data analysis, and data exploration are carried out in order to extract some meaningful insights out of the dataset.

In general, this project should prepare the dataset to be used in additional data mining and machine learning, specifically, to develop a successful intrusion detection system.

1.1 Aims and Objective

Aim:

The main aim of this coursework is to analyze and explore network traffic data using Python programming and data analysis techniques in order to understand network flow characteristic and prepare the dataset for further intrusion detection and machine learning applications.

Objective:

- To understand the structure and features of the network traffic dataset.
- To perform statistical analysis on network traffic features using Python libraries.
- To visualize network traffic patterns using charts and graphs for better understanding of normal and attack traffic.
- To compare the characteristic of Normal traffic and Attack categories through data exploration techniques.
- To prepare clean and organized data suitable for future machine learning and intrusion detection system development.
- To preprocess the dataset by handling missing values, removing duplicate record, and selecting important attributes for analysis.

1.2 Tools and Technologies used

1.2.1 Programming Language:

1. Python

Python is a general-purpose, versatile and powerful programming language. It is frequently used in the field of data analysis, machine learning, artificial intelligence, web development and scientific computing. Its flexibility and ease of development, and scientific computing due to the easy syntax and reusable code along with its many useful libraries for efficient programming (w3 school, 2026).

In this coursework, python programming languages was chosen for network traffic dataset analysis and pre-processing. Libraries such as Pandas for data manipulation. NumPy for numerical computation, Matplotlib and Seaborn for data visualization, and SciPy for statistical computations which are very powerful which makes the coursework to finish in deadline and helps in efficiently handling large datasets, performing statistical calculations, and generating graphical representation of data.

Python is also very well-suited for cybersecurity and intrusion detection research due to its ability to automate processes and support data mining and machine learning programs. It is thus perfect for analyzing network traffic data and further preparation for smart data discovery.



Figure 1: Python logo

1.2.2 Libraries Used

1. Pandas

Pandas is an open-source python library used for data manipulation and data analysis. It also offers robust, flexible data structures like Series and DataFrame, enabling users to efficiently organize, clean, filter and analyze large amount of data.

In this coursework, Pandas was used to load network traffic data into a DataFrame, which is necessary for pre-processing and analysis. It helps in handling missing values, checking for duplicate records, selecting columns of interest, renaming labels, computing summary statistics and in the exploration of data.

The versatility of Pandas makes it a popular tool in data science and machine learning, as it streamlines complex data tasks and enhances data analysis workflows. It seamlessly integrates with other popular Python libraries like NumPy, Matplotlib, and Seaborn making it incredibly well suited for applications of smart data discovery and network traffic analysis (tutorials_points , 2026).



Figure 2:pandas' logo

2. NumPy

NumPy is an open-source library written in Python that offers features for computing numbers and conducting mathematical operations. The library enables the manipulation of multidimensional arrays and matrices along with various mathematical functions that can conduct computations in an efficient manner (geeksforgeeks, 2026) .

The use of the NumPy library is significant in the domain of data science and machine learning as it enhances the speed of computation. Moreover, the library supports other libraries such as Pandas, Matplotlib, and SciPy.



NumPy

Figure 3: NumPy logo

3. Matplotlib

Matplotlib is an open-source data visualization toolkit written in Python programming language. It helps to create several kinds of plots like charts, graphs, histograms, pie chart, boxplot, among others to assist users in understanding their data by visualization (geeksforgeeks, 2026).

In the course work, Matplotlib was used to present network traffic data using bar charts, pie charts, and boxplots to identify relationships and compare statistical data regarding the network traffic. This helps the user understand how each category of traffic data can be compared to others based on statistics.

Matplotlib is a common library in data science because it makes it possible for one to present data to others in an easy way for them to comprehend. It can be easily integrated with Pandas and NumPy, thus making it useful in analyzing data.



Figure 4:matplotlib logo

4. SciPy

SciPy refers to a free, open-source Python library for scientific and statistical computation and developed in the open GitHub. This is due to its capability to provide advanced mathematical operations, statistical computation, optimizations, and hypothesis testing techniques which can be utilized for the analysis of various datasets (geeksforgeeks, 2026).

The use of SciPy was applied in this coursework where it played a key role in performing statistical tests to determine if there was any significant difference between the mean Flow Duration values between Normal Traffic and Attack classes.

SciPy is commonly applied in data science and machine learning fields due to the extended functionalities it offers compared to the NumPy library for scientific computing purposes. This makes it an ideal tool for carrying out smart data discoveries and network traffic analysis.



Figure 5:SciPy logo

5. Seaborn

The Seaborn module is an open-source library for Python that runs atop the Matplotlib library and that allows creating beautiful statistical data visualizations. Using this library, one can easily produce appealing and informative graphs in a very short amount of time compared to using other libraries for this purpose.

In this coursework, the Seaborn package has been utilized to visualize the network traffic data in the form of box plots and correlation matrices that have greatly helped to explore the dataset and understand various patterns and dependencies.

Seaborn is quite popular among those working in the area of data science and machine learning as it allows making complicated statistical graphs in a more convenient way and, at the same time, improving the visual presentation of graphs.



Figure 6:Seaborn logo

.

2 Data Understanding

Data discovery is the process of exploring and learning about a dataset to determine its structure, patterns, and other important features before proceeding to detailed analysis or constructing models. It is one of the crucial initial steps in data analysis as it assists in determining issues in the quality of data. In this project, data discovery is carried on a network traffic dataset which has diverse features like destination port, length of flow, number of packets, length of packets and traffic labels (Normal Traffic or Attack). The primary aim of this phase is to explore the dataset to understand how the various variables operate and how they interrelate.

There are multiple activities performed during the data discovery, which include investigating the dataset structure, finding the important columns, handling the presence of missing and duplicates, as well as analyzing the simple statistical properties. Bar charts, box plots and pie charts are also the methods of visualization that help to understand the distribution of the data better and identify the imbalance in the target variable. The process helps in detecting meaningful patterns, anomalies, and prepares the dataset to the subsequent processes like cleaning and transforming data and building machine learning models in detecting intrusions.

2.1 Dataset description

Table 1: Dataset table

S.N.	Column Name	Description	Data Type
1	Destination Port	The network port on the receiving device.	Integer
2	Flow Duration	Total time the network flow was active.	Integer
3	Total Fwd Packets	Total number of packets sent from the source to the destination.	Integer
4	Total Backward Packets	Total number of packets sent from the destination back to the source.	Integer
5	Total length of Fwd Packets	Aggregated size (in bytes) of all packets sent from source to destination.	Integer
6	Total length of Bwd Packets	Aggregated size (in bytes) of all packets sent from destination to source.	Integer
7	Fwd Packet Length Mean	Average size (in bytes) of packets traveling from source to destination.	Float

8	Bwd Packet Length Mean	Average size (in bytes) of packets traveling from destination to source.	Float
9	Flow Bytes /s	Rate of data transfer, measured in bytes per second.	Float
10	Flow Packets /s	Rate of packet transmission, measured in packets per second.	Float
11	Packet Length Mean	Average size (in bytes) of all packets within the flow, regardless of direction.	Float
12	Packet Length Std	Standard deviation of packet sizes, indicating variability within the flow/	Float
13	Average Packet Size	The mean packet length calculated across the entire flow duration.	Float
14	Active mean	Average time the flow was actively transferring data	Float
15	Idle Mean	Average time the flow was inactive before terminating.	Float
16	Label	Classification of traffic.	String

3 Data Preparation

Data preparation is the process of cleaning and organize raw data into a form that is appropriate to analyze. It includes the process of choosing the appropriate features, managing missing values, eliminating redundancy of records, and maintaining consistency and reliability of the dataset. In this project, the data will be loaded into a tabular format and only the relevant columns will be selected to carry out additional analysis. Missing and duplicate values are eliminated, so that the quality of the data can be maintained. Also, the categorical values present in the dataset have been changed into more significant labels to enhance understanding (techtarget , 2026)

In general, data preparation makes the dataset clean, well-structured, and prepared to conduct an effective data analysis and exploration.

3.1 Importing Datasets

```
2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind
```

Figure 7:Importing libraries

Code description:

In this step necessary Python libraries like pandas, NumPy, matplotlib, and seaborn are imported that will aid in data analysis. Pandas is employed to load and manipulate the data in a structured format whereas NumPy assists in making numerical calculations. Visualizations such as charts and graphs are created using Matplotlib and seaborn.

3.2 Displaying the last 50 records

```
#Loading the data set and displaying last 50 records
df = pd.read_csv('ids.csv')
df.tail(50)
```

Figure 8:Code to display 50 records

Code description:

After successfully importing pandas, we gain access to many useful function which helps us in understanding, preparation, analysis etc. Like that one of the most commonly used commands is read_csv() function which helps in utilizing to read the dataset file and store it in a tabular

format in a structured manner. Once the dataset is loaded, the `tail()` function is used to display records of the DataFrame. As we need to display the last 50 records so we write 50 inside the `tail()` function parenthesis.

3.3 Creating a separate DataFrame with selected columns.

```
[ ]: # selecting some columns from the original DataFrame
selected_columns = ['Destination Port', 'Flow Duration', 'Total Fwd Packets', 'Total Backward Packets',
                    'Total Length of Fwd Packets', 'Total Length of Bwd Packets',
                    'Fwd Packet Length Mean', 'Bwd Packet Length Mean', 'Flow Bytes/s',
                    'Flow Packets/s', 'Packet Length Mean', 'Packet Length Std',
                    'Average Packet Size', 'Active Mean', 'Label']

# new dataframe with some selected column
new_df = df[selected_columns]

# displaying the rows
new_df.head()
```

Figure 9: Python code code of separate Dataframe with selected columns

Code description:

In this step, a new DataFrame is created by selecting the necessary columns in the original dataset. This will helps in eliminating irrelevant information and highlights the the significant features to be analyzed. The selected columns chosen contain the important network flow attributes like the numbers of packets, the length of packets, flow statistics and the Label column which is useful in determining whether the traffic is normal or an attack. These characteristics can help us learn the patterns and relationships within the data.

The names of columns are specified in a list and pandas indexing is applied to retrieve these columns into a new DataFrame. This will enable the dataset to be more organized and manageable and be used in subsequent data preparation and analysis. And lastly pandas function `head()` is used to display five rows of the selected columns.

Output:

[7]:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size	Active Mean
0	53	62171	2	2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000	0.000000
1	2710	48	2	0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000	0.000000
2	443	4792909	5	1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333	0.000000
3	80	115596470	75	82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962	22092.63636
4	2910	14	2	2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000	0.000000

Figure 10:Output of showing DataFrame with selected columns

3.4 Handling Missing values (NaN)

```
# checking and removing NaN values

new_df.isnull()

# Count NaN values in each columns
print(new_df.isnull().sum())

|
```

Figure 11: Python code of checking handling value

Code description:

In this step, isNull() function is used on new_df to identify missing data in the selected DataFrame. It returns True when values is missing and give false where it is not. Then isnull().sum() function is used to determine how many missing (NaN) values there are in each column of new_df, provide column-by-column count of missing values.

Output:

```
Destination Port      0
Flow Duration         0
Total Fwd Packets     0
Total Backward Packets 0
Total Length of Fwd Packets 0
Total Length of Bwd Packets 0
Fwd Packet Length Mean 0
Bwd Packet Length Mean 0
Flow Bytes/s         36
Flow Packets/s        0
Packet Length Mean    0
Packet Length Std     0
Average Packet Size   0
Active Mean           0
Idle Mean             0
Label                0
dtype: int64
```

Figure 12: Output of handling value

After identifying the missing values the `dropna()` function is applied on `new_df` to drop the rows which have NaN values. Then the cleaned data is stored in a new DataFrame called `new_cleaned_df`. Lastly, to ensure that they do not have any missing value left, `isnull().sum()` is called once more on `new_cleaned_df`, which is supposed to have zero values on all columns.

```
# removing Nan values

new_cleaned_df = new_df.dropna()
print(new_cleaned_df.isnull().sum())
```

Figure 13: Python code of removing NaN values

Output:

```
Destination Port      0
Flow Duration         0
Total Fwd Packets     0
Total Backward Packets 0
Total Length of Fwd Packets 0
Total Length of Bwd Packets 0
Fwd Packet Length Mean 0
Bwd Packet Length Mean 0
Flow Bytes/s         0
Flow Packets/s       0
Packet Length Mean   0
Packet Length Std    0
Average Packet Size  0
Active Mean          0
Idle Mean            0
Label               0
dtype: int64
```

Figure 14: Output of showing removing Nan values

3.5 Checking Duplicate Records

```
# Checking Duplicate Records

new_cleaned_df.duplicated()
duplicate_count = new_cleaned_df.duplicated().sum()
print('Total number of duplicate records:', duplicate_count)
```

Figure 15: Python code of checking duplicate records

Code description:

In this step, the `duplicated()` function is applied on `new_cleaned_df` to check whether any duplicated records exist in the DataFrame. It returns a Boolean value for each row, whether it is a duplicated of a previous row. Lastly the `sum()` function is used to find the total number of duplicate records which counts all the True values.

Output:

```
Total number of duplicate records: 139389
```

Figure 16: Output of showing total number of duplicate records

3.6 Displaying total number of columns and column names.

```
# Showing number of columns and displaying them
total_columns = new_cleaned_df.shape[1]
print(f'Total number of columns: {total_columns}')

column_list = list(new_df.columns)

print('Name of the columns is given by:-')
for i, name in enumerate(column_list, start=1):
    print(f'{i}. {name}')
```

Figure 17: Python code of displaying total number of columns and column names

Code description:

In this step, the total number of column and names were displayed to confirm structure of the filter DataFrame. The shape[1] attribute used to retrieve total number of columns and stored in variable named total_columns. Then the value was printed to show the exact number of columns present in dataset. With the help of list() function to convert the column names into a list stored in column_list, and to display in a clear and in more readable format, Lastly for loop with the enumerate() function is used to iterate each column.

Output:

```
Total number of columns: 16
Name of the columns is given by:-
1. Destination Port
2. Flow Duration
3. Total Fwd Packets
4. Total Backward Packets
5. Total Length of Fwd Packets
6. Total Length of Bwd Packets
7. Fwd Packet Length Mean
8. Bwd Packet Length Mean
9. Flow Bytes/s
10. Flow Packets/s
11. Packet Length Mean
12. Packet Length Std
13. Average Packet Size
14. Active Mean
15. Idle Mean
16. Label
```

Figure 18: Output of displaying total number of columns and column names

3.7 Renaming Label name

```
### Renaming Label values

print("Label values before renaming:")
print(new_cleaned_df['Label'].unique())

new_cleaned_df.loc[:, 'Label'] = new_cleaned_df['Label'].replace({
    'BENIGN': 'Normal Traffic',
    'Infiltration': 'Attack'
})

print("\nLabel values after renaming:")
print(new_cleaned_df['Label'].unique())
```

Figure 19: Python code to rename label name

Code description:

In this step, the values in the label column were renamed so that to make them meaningful. The `unique()` function is used before renaming to display the original label. And the `replace()` function is used to rename BEGIN to Normal Traffic and Infiltration to Attack. And `loc[ ]` is used to avoid pandas warnings while modifying column names. And lastly `unique()` function was used after renaming to confirm the label values were successfully updated.

Output:

```
Label values before renaming:
['BENIGN' 'Infiltration']

Label values after renaming:
['Normal Traffic' 'Attack']
```

Figure 20: Output of renaming label name

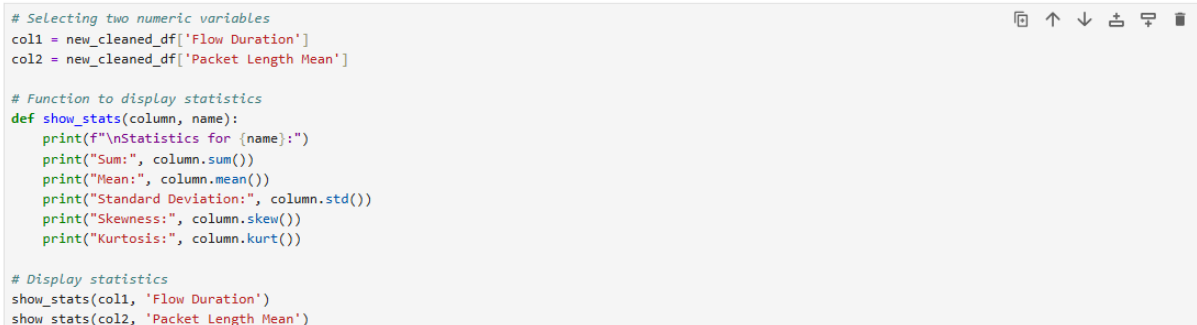
4 Data Analysis

Data analysis is an activity that involves the analysis of the prepared data with the aim of deriving meaningful information and interpretation of trends in the data. It helps in identifying the relationships between variables and generalizing the main characteristics of the data set.

In this project the analysis of data is done on a statistical basis whereby some of the numerical variables are analyzed on the basis of the following statistical measures such as sum, mean, standard deviation, skewness, and kurtosis. These measures provide details on the distribution, central tendency and variability of the data. Also, correlation analysis will be done to determine the associations among various numerical characteristics. This assists in determining the degree of association of variables with each other and in choosing significant factors to be further analyzed or to construct a model (datacamp, 2026).

In general, data analysis can assist in getting a deeper understanding of the data and make better decision-making action like data exploration.

4.1 Summary Statistics of selected two numeric values.



```
# Selecting two numeric variables
col1 = new_cleaned_df['Flow Duration']
col2 = new_cleaned_df['Packet Length Mean']

# Function to display statistics
def show_stats(column, name):
    print(f"\nStatistics for {name}:")
    print("Sum:", column.sum())
    print("Mean:", column.mean())
    print("Standard Deviation:", column.std())
    print("Skewness:", column.skew())
    print("Kurtosis:", column.kurt())

# Display statistics
show_stats(col1, 'Flow Duration')
show_stats(col2, 'Packet Length Mean')
```

Figure 21: Python code to display summary Statistics

Code description:

The step involves the calculating of summary statistics of two selected numerical variables to understand their distribution and their main characteristics. Statistical values like sum, mean, standard deviation, skewness, and kurtosis are useful in examining the central tendency of the data, data spread, and shape. Statistical value sum shows total value of the variable, and mean represents the average. The standard deviation describe the variation or dispersion of the data around the mean. The skewness is a used to measure the asymmetry of the data distribution where distribution is asymmetrical when its left and right side are not mirror image. Lastly the

kurtosis is used to measure the tailedness of the distribution. Such statistics provide a better understanding of how the selected variables behave and can be used to find any suspicious trends or patterns in the dataset.

Output:

```
Statistics for Flow Duration:
Sum: 4364852868645
Mean: 8942922.905195676
Standard Deviation: 27481820.041128952
Skewness: 3.2508743768773116
Kurtosis: 9.246790962992351

Statistics for Packet Length Mean:
Sum: 41757628.631677166
Mean: 85.55506102839328
Standard Deviation: 173.87246683072507
Skewness: 4.935765643058977
Kurtosis: 34.61051191591095
```

Figure 22: Output of summary statistics

4.2 Correlation analysis of numeric values

```
# Correlation matrix (numeric only)
correlation_matrix = new_cleaned_df.corr(numeric_only=True)

# Top 5 correlated features
corr_pairs = correlation_matrix.unstack()
sorted_pairs = corr_pairs.sort_values(ascending=False)

# Remove duplicate/self correlations
top_5 = sorted_pairs.drop_duplicates().iloc[1:6]

print("\nTop 5 Correlated Features:")
print(top_5)
```

Figure 23: Python code to display correlation analysis of numeric values

Code description:

Correlation analysis is carried out in this step to establish the strongest relationships between numerical variables in dataset. The correlation matrix is initially calculated with the help of numeric columns only to make sure that the results are correct. The `unstack()` function is then used to convert the matrix into pairs of correlations of features. These pairs are ranked in descending order of their correlation values, so that the largest correlations are found at the beginning. Since the correlation matrix contains pairs of variables that are the same (duplicate correlations) and pairs of variables that are the same with themselves (self-correlations), these have been eliminated to eliminate redundancy. Lastly the 5 most strongly correlated feature pairs are chosen and shown.

Output:

```
Top 5 Correlated Features:
Packet Length Mean      Average Packet Size      0.995068
Total Length of Bwd Packets  Total Backward Packets  0.961480
Total Fwd Packets        Total Backward Packets  0.949848
                        Total Length of Bwd Packets  0.920007
Packet Length Mean      Packet Length Std        0.887792
dtype: float64
```

Figure 24: Output showing a correlation analysis

5 Data Exploration

The process analyzing and visualizing the data to understand its format, structure, patterns and the relationship among variables is Data exploration. It helps in developing in-depth understanding or insights of the data and also supports in making better decisions to develop further analysis. In this project, data exploration has been carried out through different visualization tools like bar charts, pie charts and boxplots. These charts can be used to see how the target variable (Label) is distributed, whether the data is balanced or skewed, and compare various categories, like normal traffic and attack traffic. Besides, the differences in features such as flow duration and packet length in various categories are analyzed with the help of a graph. The distribution and dispersion of data can be monitored with boxplots, whereas frequencies and proportions can be compared with the help of charts such as bar graphs and pie charts (geeksforgeek, 2025).

Overall, data exploration allows the dataset to be interpreted or understandable better by converting numerical data into visual information or insights, which helps in determining or identifying patterns, trends, and anomalies to be analyzed further.

5.1 Bar chart visualization and Balance Analysis of Label

```

# Frequency count of each category
label_counts = new_cleaned_df['Label'].value_counts()

print("Frequency of Label categories:")
print(label_counts)

# Bar chart visualization
plt.figure(figsize=(6,4))
sns.barplot(x=label_counts.index, y=label_counts.values, hue=label_counts.index, palette=["#1f77b4", "#ff7f0e"], legend=False)
plt.title("Frequency of Label Categories")
plt.xlabel("Label")
plt.ylabel("Count")
plt.show()

# Balance check
normal_count = label_counts.get("Normal Traffic", 0)
attack_count = label_counts.get("Attack", 0)

if abs(normal_count - attack_count) < 0.1 * (normal_count + attack_count):
    print("The dataset is relatively balanced between Normal Traffic and Attack.")
else:
    print("The dataset is imbalanced. One category has significantly more samples than the other.")

```

Figure 25: Python code to visualize bar chart and balance analysis of label

Code description:

In this above code, the frequency of each of the categories in the Label column is calculated and represents a bar chart. The number of times each category, namely “Normal Traffic” and Attack appear is counted using the `value_counts()` function. Then the frequency of each category is compared visually through the use of the seaborn library to create a bar chart and in that bar chart different colors are used to clearly distinguish between the categories, making the visualization more understandable so that the chart helps in analysis easily. The chart also includes some appropriate titles and axis labels to make it well presented.

Once it has been visualized, a balance check is done to check whether the dataset is balanced or imbalanced. The counts of the "Normal" Traffic and the "Attack" are obtained and the difference between the numbers is compared with 10 percent of the total records and when if the difference is small, then the dataset is said to be relatively balanced, otherwise it is said to be imbalanced.

Output:

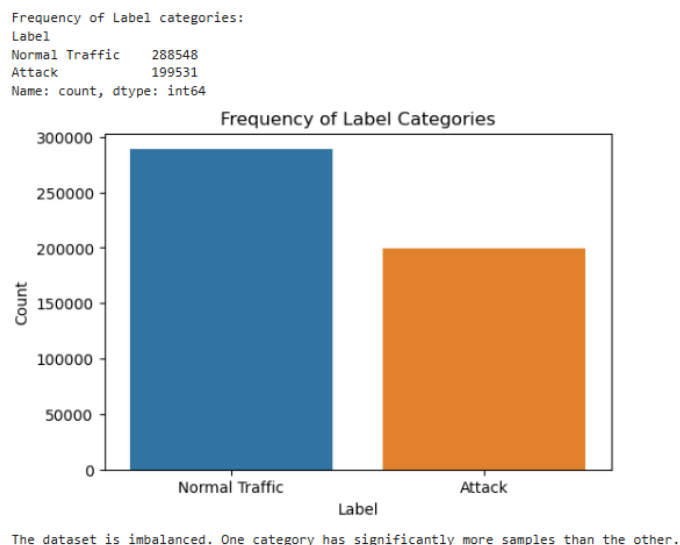


Figure 26: Output of showing visualize bar chart and balance analysis of label

Bar Chart description:

The bar chart shows the distribution of the dependent variable Label that has two categories: Normal Traffic and Attack. From the graph, it is observed that the blue bar Normal Traffic is higher than the orange bar Attack as there are 2,88,548 records and 1,99,531 records. This difference shows that the dataset is imbalanced as Normal Traffic is considered more samples than the Attack samples. Such imbalance can affect the performance or to the conduct of machine learning models, which might be biased towards the majority group

5.2 Analysis of Average Flow Duration and Packet Length Mean by Label Categories

```
# Calculate average Flow Duration and Packet Length Mean by Label
avg_values = new_cleaned_df.groupby('Label')[['Flow Duration','Packet Length Mean']].mean()

print("Average values by Label:")
print(avg_values)

# Determine which category has the highest averages
highest_flow = avg_values['Flow Duration'].idxmax()
highest_packet = avg_values['Packet Length Mean'].idxmax()

print("\nHighest average Flow Duration:",highest_flow)
print("Highest average Packet Length Mean:",highest_packet)

# Visualization with Pie Charts

plt.figure(figsize=(12,5))

# Pie chart for Flow Duration
plt.subplot(1,2,1)
plt.pie(avg_values['Flow Duration'], labels=avg_values.index, autopct='%1.1f%%', colors=['#1f77b4','#ff7f0e'])
plt.title("Average Flow Duration by Label")

# Pie chart for Packet Length Mean
plt.subplot(1,2,2)
plt.pie(avg_values['Packet Length Mean'], labels=avg_values.index, autopct='%1.1f%%', colors=['#1f77b4','#ff7f0e'])
plt.title("Average Packet Length Mean by Label")

plt.show()
```

Figure 27: Code to display average flow duration and packet length mean

Code description:

In the above code, the average value of Flow Duration and Packet Length Mean are computed in each category of the Label column (Normal Traffic and Attack). The groupby() function is used to do this by grouping the data by the Label and the mean of the chosen features is calculated. Pie charts are then used to visualize the results to compare the contribution of each category. Two pie charts are generated one of average Flow Duration and another of average Packet Length Mean. The charts shows the proportion of each of the categories making less difficult to see which category contains more average values than the other one.

Output:

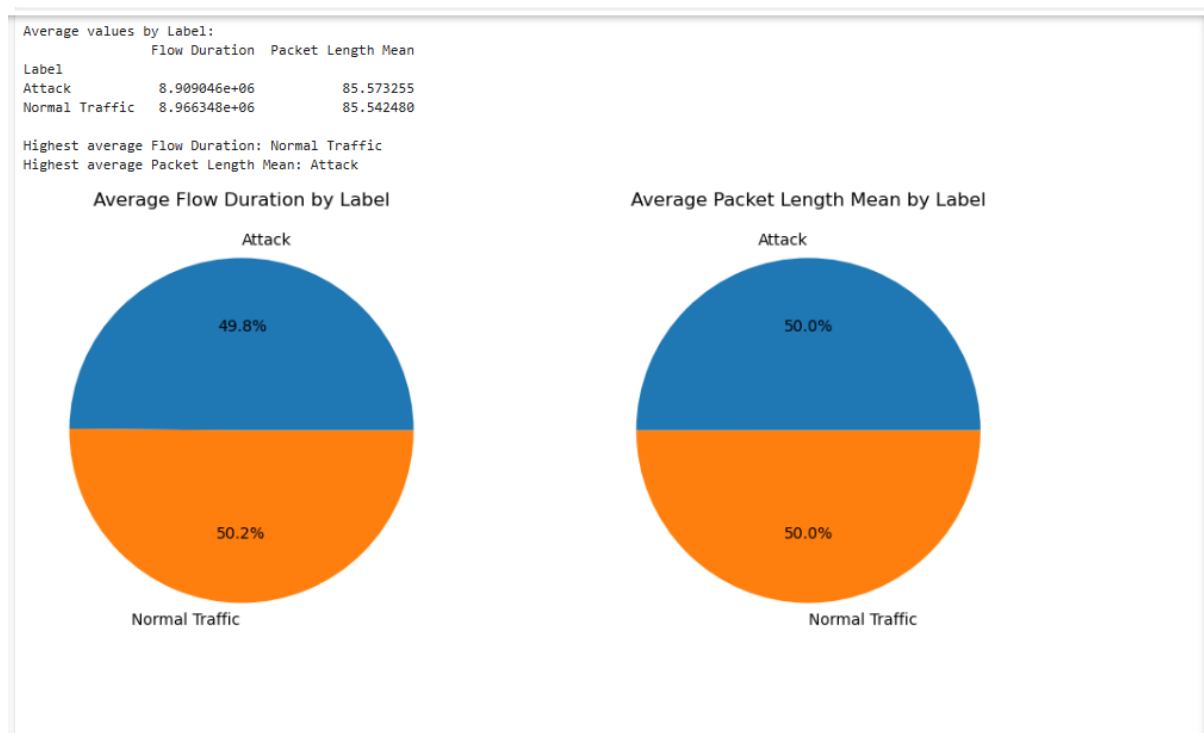


Figure 28: Output showing average flow duration and packet length mean

Pie Chart Description:

The pie charts show the means of Flow Duration and Packet Length Mean to the two classes in the Label column: Normal Traffic and Attack. Each chart is the proportion contribution of each category to the average value.

Based on the first pie chart Average Flow Duration, one can be observed that the average Flow Duration of the Normal Traffic is slightly larger (50.2%) than that of Attack (49.8%). This means that normal traffic flows are likely to be slightly longer than attack traffic.

Conversely, the second pie chart (Average Packet Length Mean) indicates that Attack traffic experiences a slightly higher average Packet Length Mean (50.0%) than the Normal Traffic (50.0%). However, the difference is very insignificant. This would imply that attack traffic would have a tendency to have a higher packet size on average.

Overall, both charts indicate that the variations or difference between the two categories are small, which means that these features alone might not significantly discriminate between

Normal Traffic and Attack. The minor differences can however be useful when used together with other features to build a model or further analysis.

5.3 Boxplot of Fwd Packet Length Mean by Traffic Type

```
# Boxplot of Fwd Packet Length Mean grouped by Label
plt.figure(figsize=(6,4))

sns.boxplot(x='Label', y='Fwd Packet Length Mean', data=new_cleaned_df,hue='Label', palette=["#1f77b4", "#ff7f0e"], legend=False)

plt.title("Fwd Packet Length Mean by Label")
plt.xlabel("Label")
plt.ylabel("Fwd Packet Length Mean")

plt.show()
```

Figure 29: Code to display Fwd Packet Length mean by traffic type

Code description:

In this step, a boxplot is created to show the distribution of Fwd Packet Length Mean of each category in the Label column (Normal Traffic and Attack). The boxplot provide an overview of the data distribution, such as the median, quartiles and the possible outliers. The x-axis shows the categories (Normal Traffic and Attack), and the y-axis is the values of Fwd Packet Length Mean. This graph assists in the comparison between the distribution of the values of both categories.

Output:

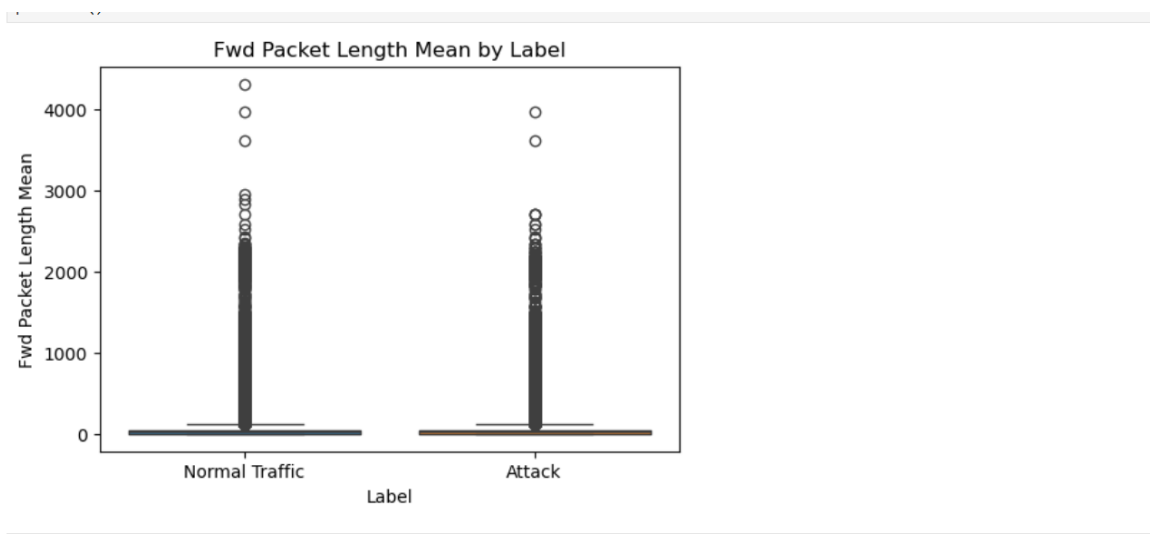


Figure 30: Boxplot graph

Analysis of Boxplot graph:

The boxplot is used to display the distribution of Fwd Packet Length Mean of both the Normal Traffic and Attack categories. As can be seen, the distribution and central tendency of the two types are different. Median values of both categories can be slightly different, which means that there is a variation of typical packet length. Also, the spread of the data (interquartile range) indicates the amount of variability each category has. Outliers are also possible, which means that there are extreme values of the lengths of packets. Such variations imply that Fwd Packet Length Mean differs between normal and attack traffic and can be used to identify the two types of traffic.

5.4 Hypothesis testing

```

1: |
# Separate the two groups
normal = new_cleaned_df[new_cleaned_df['Label'] == 'Normal Traffic']['Flow Duration']
attack = new_cleaned_df[new_cleaned_df['Label'] == 'Attack']['Flow Duration']

# Perform independent t-test
t_stat, p_value = ttest_ind(normal, attack, equal_var=False)

print("T-Statistic:", t_stat)
print("P-Value:", p_value)

# Decision
if p_value < 0.05:
    print("Result: Reject the null hypothesis (significant difference exists).")
else:
    print("Result: Fail to reject the null hypothesis (no significant difference).")

```

Figure 31: Code of hypothesis testing

Code description:

In this step, a hypothesis test is conducted to find out whether there is significant difference in mean Flow Duration in normal traffic and Attack categories. The reason why an independent t-test is used is that the two independent groups are compared. The dataset is separated into two categories according to the column of Label and the values of Flow Duration are taken out of each category.

Output:

```

T-Statistic: 0.7166329663217962
P-Value: 0.47360098679301743
Result: Fail to reject the null hypothesis (no significant difference).

```

Figure 32: Output of hypothesis testing

6 Conclusion

In this project, data on network traffic was examined through the application of different data discovery, preparation, analysis and exploration tools. The initial stage of analyzing the data involved cleaning and organizing it by addressing missing values, eliminating duplicates, and the selection of features of interest to the subsequent analysis.

Correlation and statistical methods were used to get insight into relationships between variables and visualization techniques including bar charts, pie charts, and boxplots were used to identify patterns and differences between categories: Normal Traffic and Attack. The analysis showed that the dataset is skewed, and the number of Normal Traffic was greater than that of Attack.

Further investigation revealed that there were minor variations in the characteristics of Flow Duration and Packet Length Mean between the two categories. The test of hypothesis helped to confirm that these differences are significant, giving a more in-depth insight into the data.

Overall, this discussion can assist in learning how network traffic operates and pre-treat the data to be utilized in subsequent machine learning tasks like intrusion detection. The results demonstrate that good data preparation and feature analysis are crucial elements of developing effective predictive models.

7 References

datacamp. (2026) *what-is-data-analysis-expert-guide* [Online]. Available from: <https://www.datacamp.com/blog/what-is-data-analysis-expert-guide> [Accessed 28 April 2026].

geeksforgeek. (2025) *https://www.geeksforgeeks.org/data-analysis/what-is-data-exploration-and-its-process/* [Online]. Available from: <https://www.geeksforgeeks.org/data-analysis/what-is-data-exploration-and-its-process/> [Accessed 28 April 2026].

geeksforgeeks. (2026) *https://www.geeksforgeeks.org/python/matplotlib-tutorial/* [Online]. Available from: <https://www.geeksforgeeks.org/python/matplotlib-tutorial/> [Accessed 06 May 2026].

geeksforgeeks. (2026) *scipy_tutorial* [Online]. Available from: <https://www.geeksforgeeks.org/machine-learning/scipy-tutorial/> [Accessed 07 May 2026].

geeksforgeeks. (2026) *python-numpy* [Online]. Available from: <https://www.geeksforgeeks.org/numpy/python-numpy/> [Accessed 06 May 2026].

techtarget. (2026) *data-preparatiion* [Online]. Available from: <https://www.techtarget.com/searchbusinessanalytics/definition/data-preparation> [Accessed 28 April 2026].

tutorials_points. (2026) *python_pandas* [Online]. Available from: https://www.tutorialspoint.com/python_pandas/index.htm [Accessed 03 May 2026].

w3 school. (2026) *python* [Online]. Available from: <https://www.w3schools.com/python/> [Accessed 01 May 2026].